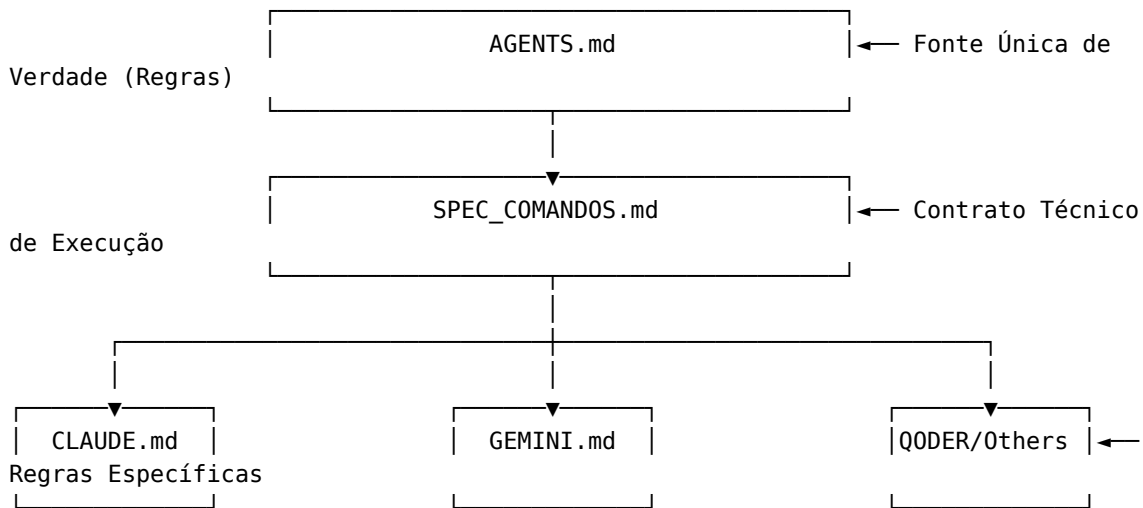


09 — Processo Completo e Arquitetura da Fábrica de Comunicação

Status: Concluído (Documentação Oficial do Sistema) **Data:** 2026-08-06 **Escopo:** Mapeamento ponta a ponta dos processos, governança de arquivos, orquestração agêntica, ferramentas (Skills, MCPs, Hooks, Specs) e scripts determinísticos.

1. Governança e Estrutura de Arquivos de Diretriz

O ecossistema da **Fábrica de Materiais de Comunicação** é regido por uma hierarquia rígida de arquivos de documentação e controle. Esse arranjo garante que as regras de negócio sejam centralizadas, as definições visuais permaneçam fixas e a fábrica opere de forma idêntica e consistente em qualquer interface de execução.



1.1. O Papel de Cada Arquivo de Diretriz

- **AGENTS.md (Fonte Única de Verdade - SSOT):** Rege as regras de arquitetura geral, governança de agentes, catálogo de módulos e as **Regras Invioláveis do Projeto** (como idioma estrito em PT-BR, silenciamento estético, autonomia pós-esboço, auto-correção interna e priorização do Grafo de Conhecimento).
- **SPEC_COMANDOS.md (Contrato de Comandos):** Define o fluxo de execução canônico de cada comando de forma universal (ex: /esbocar, /produzir-comunicacao-completa, /gerar-pdf, etc.). Qualquer harness executa exatamente os procedimentos descritos neste arquivo.
- **CLAUDE.md, GEMINI.md, CODEBUDDY.md, QODER.md (Rules Específicas por Harness):** Arquivos de regras finas carregados dinamicamente por cada assistente. Eles referenciam AGENTS.md e SPEC_COMANDOS.md sem duplicar sua lógica interna, adicionando apenas instruções específicas de formato do console ou configurações de MCP nativos de cada interface.

1.2. A Arquitetura de 3 Camadas para Universalidade Total (REGRA 10)

Para suportar qualquer executor (Claude Code, opencode, Gemini CLI, CodeBuddy, Qoder, etc.), o projeto implementa uma divisão em três camadas:

1. **Camada 1 (Canônico Único):** Regras de negócio, especificações visuais, procedimentos e skills agênticas. Vivem em `AGENTS.md`, `SPEC_COMANDOS.md`, `brand/` e `.claude/skills/*/SKILL.md`. Não há duplicação dessa lógica.
 2. **Camada 2 (Adaptadores Finos de Descoberta):** Arquivos leves específicos de cada ferramenta para mapeamento de autocomplete de comandos, declaração de MCPs ou hooks nativos. Ex: `.claude/commands/*.md`, `.opencode/commands/*.md`, `.mcp.json` e `opencode.jsonc`. Eles atuam puramente como ponteiros de execução para a Camada 1.
 3. **Camada 3 (Rules Finas por Harness):** Arquivos de configuração rápidos carregados pelas ferramentas no início das sessões (ex: `CLAUDE.md`, `GEMINI.md`, `CODEBUDDY.md`, `QODER.md`).
-

2. Visão Geral da Estrutura de Diretórios

O repositório é organizado de forma a separar estritamente as regras de marca, templates, ferramentas de orquestração (skills agênticas e scripts python) e os resultados gerados por projeto:

- **assets/:** Contém as fontes visuais de marca (logos em preto, branco, horizontal e vertical) e os backgrounds base de template (`modelo-1080x1080.png`, etc.).
 - **brand/:** Guarda as definições estáticas do Design System fixo da marca Conexão (`design-system-conexao.json`, `kits-conexao.json`, etc.) — que regem cores, gradientes e públicos de forma imutável por projeto.
 - **docs/:** Armazena relatórios de auditoria, melhorias estruturais, planos de expansão e este manual de processo técnico do ecossistema.
 - **templates/:** Contém as estruturas base para compilação (arquivos HTML e templates Typst para apostilas, além de fontes WOFF2 embutidas).
 - **scripts/:** O coração determinístico do repositório. Centraliza os árbitros de qualidade (`validar-*.py`, `auditar-projeto.py`), o script de orquestração (`pool-materiais.py`) e os compiladores de código final (`compilar-*.py`).
 - **.claude/ (ou similares de outros harnesses):** Centraliza os comandos mapeados de UI e, principalmente, as **Agent Skills** (habilidades agênticas injetadas na sessão que orientam a redação e validação do pipeline).
 - **output/<slug>/:** Pasta isolada por projeto (identificado por seu slug), contendo todas as definições estruturais (`config_projeto.json`, `_pool_estado.json`, `brief_criativo.json`), arquivos intermediários de redação e as pastas finais limpas de cada material compilado (ex: `pdf/`, `landing-page/`, `apresentacao/`, etc.).
-

3. O Fluxo de Processo Completo (Passo a Passo)

A fábrica opera em um fluxo de **2 Passos principais**, divididos entre a captura humana controlada e a produção autonomizada assistida por gates de validação rígidos.

[PASSO 1: INTERAÇÃO HUMANA]

↓
▼
/esbocar <argumentos>
↓

1. *Rodada 1 (Insumos)*: Captura do nome do projeto, links ou paths das imagens de produto/marca e o texto-base de conteúdo (fonte absoluta e única de Claims técnicos).
 2. *Rodada 2 (Público-Alvo)*: Escolha estruturada (Consultores, Clientes ou Distribuidores).
 3. *Rodada 3 (Objetivo/Tom)*: Definição do tom do material (Educativo/Comercial, Informativo/Técnico ou Comercial/Informativo de Parceria).
 4. *Rodada 4 (Materiais)*: Seleção múltipla de quais materiais serão gerados (PDF, Landing Page, Apresentação, Artes PNG, Textos de Apoio ou Kits).
- **Pós-Processamento Automático**: Assim que a rodada 4 termina, o comando grava o `config_projeto.json` e dispara de forma imediata e silenciosa (sem interação humana):
 - A Skill **analista-insumos**: Organiza as matérias-primas e gera o `dossie_insumos.md`.
 - A Skill **diretor-de-arte**: Decompõe a escolha de tom do operador, analisa o dossiê e gera o `brief_criativo.json`.
 - **Ferramentas Utilizadas**:
 - *Skills*: `analista-insumos`, `diretor-de-arte`.
 - *Mecanismo UI*: `AskUserQuestion` (Claude Code) ou questionário de texto estruturado equivalente (outros harnesses).
 - *Validador*: `scripts/parametros_projeto.py --validar`.
-

Etapa 2: Produção e Orquestração (/produzir-comunicacao-completa)

- **O que faz**: Produz todos os materiais selecionados no passo anterior de forma paralela e estritamente autônoma. Se ocorrerem erros visuais ou de conteúdo, a própria fábrica se encarrega de aplicar a auto-correção interna antes de liberar o relatório de conclusão.
- **Subetapas Técnicas**:
 1. **Fase de Validação de Setup**: Roda o `parametros_projeto.py` para garantir que o arquivo de configuração do projeto está estruturalmente perfeito antes de gastar qualquer token.
 2. **Geração de Cópies Compartilhadas (se houver Artes ou Kits selecionados)**: Caso materiais do tipo `arte-0N` ou `kit-*` sejam selecionados, a fábrica executa a Skill `redator-arte` ou `redator-kit-copy` para produzir as cópias estruturadas em arquivos JSON (`arte/copies.json` ou `kits/copies.json`) *uma única vez* de forma centralizada. Isso garante coesão contextual entre as diferentes dimensões ou públicos de kit.
 3. **Fan-Out em Lote (Orquestração do Pool)**: O script `pool-materiais.py` monta a fila de materiais pendentes e dispara lotes paralelos de tamanho máximo de 4 (`LOTE_PADRAO = 4`). Isso impede gargalos de limite de tokens por minuto (TPM) e facilita o rastreamento individual.
 4. **Execução das Unidades de Trabalho (Materiais)**: Para cada material em processamento, o pipeline agêntico:
 - Dispara o subagente produtor específico (ex: `subagente-produtor-landing`, `subagente-produtor-pdf`).
 - Cada subagente invoca a Skill de redação adequada (ex: `redator-landing`, `redator-apostila`) para escrever o roteiro ou código estrutural básico.
 - Roda o compilador específico em `scripts/` (ex: `compilar-html.py`, `compilar-pdf.py`, `compilar-arte.py`) para renderizar a saída real de design.

- Executa o script de validação de qualidade correspondente (ex: `validar-html.py`, `validar-pdf.py`, `validar-dimensoes.py`).
- 5. **Máquina de Estados de Auto-Correção:** Se o validador falhar (exit code 1), o material volta ao estado de `em_producao` para re-geração estruturada (máximo de 3 tentativas com backoff exponencial antes de transitar para esgotado). Se passar, vai para `aguardando_revisao`.
- 6. **Revisão Geral de Marca:** A Skill `revisor-marca` analisa o conjunto consolidado de saídas comparando as declarações geradas com o `brief_criativo.json` e o texto-base original (REGRA 6 — Sem claims inventados).
- 7. **Gate Determinístico Final:** O script `auditar-projeto.py --estrito` é executado sobre a pasta do projeto. Ele atua como árbitro matemático intransigente: verifica se todas as dimensões de PNG estão milimetricamente corretas, se o tamanho dos arquivos PDF respeita os limites físicos do projeto, se todas as cores e fontes utilizadas batem rigorosamente com o `brand/design-system-conexao.json` e se não há caminhos relativos quebrados.
- 8. **Empacotamento e Manifesto:** O script `empacotar-projeto.py` consolida os diretórios finais sob `output/<slug>/` e escreve o `manifesto_materiais.json` detalhando: as decisões visuais tomadas, as informações identificadas como faltantes na fonte original que precisam de complemento humano e sugestões otimizadas de legendas/CTAs.
- **Ferramentas Utilizadas:**
 - *Scripts de Validação:* `validar-*.py`, `auditar-projeto.py`, `verificar-consistencia-pipeline.py`, `verificar-universalidade.py`.
 - *Scripts Compiladores:* `compilar-html.py`, `compilar-pdf.py`, `compilar-arte.py`, `compilar-kit.py`, `pdf_typst.py`.
 - *Skills de Redação/Produção:* `redator-apostila`, `redator-landing`, `redator-apresentacao`, `redator-arte`, `redator-kit-copy`, `revisor-marca`, `aplicador-marca-conexao`.

5. Ferramentas, Skills, MCPs e Hooks Detalhados

A Fábrica de Comunicação integra recursos dinâmicos (Skills), ferramentas de contexto de grafo (MCPs), automações do ciclo de vida da sessão (Hooks) e arquivos de design/procedimento fixos (Specs).

5.1. Agent Skills (Skills Ativas)

As Skills são módulos de comportamento e instrução especializados, declarados de forma declarativa e portátil (conforme o padrão Agent Skills) na pasta `.claude/skills/`.

Nome da Skill	Objetivo Primário	Escopo de Execução
analista-insumos	Processar textos brutas e imagens do operador	Etapa 1 (/esbocar pós-rodada 4)
diretor-de-arte	Decompor tom e planejar a direção visual	Etapa 1 (/esbocar pós-rodada 4)
aplicador-marca-conexao	Injetar regras estritas de DS (cores/gradientes/badges)	Etapa 2 (Compilação HTML/Arte/Kits)
redator-apostila	Redigir apostila estruturada em Markdown para PDF	Etapa 2 (Produção da Apostila)
redator-landing	Estruturar HTML semântico e seções de conversão	Etapa 2 (Produção da LP)
redator-apresentacao	Projetar roteiro de slides em HTML (listas de marcadores)	Etapa 2 (Produção da Apresentação)
redator-arte	Escrever copies e conceitos visuais para artes PNG	Etapa 2 (Produção de Artes)
redator-kit-copy	Escrever 10 copies segmentadas (WhatsApp + redes)	Etapa 2 (Produção de Kits)
revisor-marca	Analisar conformidade estética e semântica pós-produção	Etapa 2 (Verificação de Lotes)

5.2. MCP (Model Context Protocol)

O projeto integra nativamente o **code-review-graph** (CRG) como seu MCP principal. Sua declaração está disponível tanto em `.mcp.json` quanto em `opencode.jsonc`.

- **Objetivo do MCP:** Mapear o grafo de conhecimento estrutural de todo o código e scripts em `scripts/*.py`. Ele permite que o assistente analise o impacto (blast radius) de alterações de código, localize dependências de funções, identifique gargalos de complexidade e busque referências sem precisar ler arquivos em sua totalidade de forma token-ineficiente (REGRA 9).
- **Comandos de MCP Chave:**
 - `semantic_search_nodes_tool`: Busca funções ou classes pelo nome ou conceito.
 - `query_graph_tool`: Executa varreduras estruturais como `callers_of`, `callees_of`, `tests_for`.
 - `detect_changes_tool`: Realiza uma análise de risco focada sobre alterações pendentes no git.
 - `get_review_context_tool`: Extrai trechos de código hiper-focados de forma token-efficient.

5.3. Hooks de Sessão e Atualização

Os hooks automatizam tarefas de conveniência integrando o ciclo de vida da sessão às ferramentas do ecossistema. Vivem em `.claude/settings.json`:

- **SessionStart:** Dispara o script `.gemini/hooks/crg-session-start.sh`. Ele constrói ou lê o cache do grafo de conhecimento do projeto no início de cada nova conversa.
 - **PostToolUse:** Dispara o script `.gemini/hooks/crg-update.sh`. Atualiza incrementalmente o grafo sempre que scripts python ou configurações são modificados, garantindo que o `code-review-graph` esteja sempre atualizado. Além disso, executa os validadores determinísticos universais (`verificar-universalidade.py --estrito` e `verificar-consistencia-pipeline.py --estrito`) para garantir que nenhuma alteração quebre a portabilidade multiplataforma da fábrica.
-

5.4. Specs (Especificações de Artefato)

Cada material produzido possui sua própria especificação técnica inviolável (Specs), atuando como contrato técnico estrito:

- **SPEC_PDF.md (Contrato do PDF):**
 - *Como funciona:* Transforma a apostila estruturada em Markdown gerada pelo subagente em um PDF pixel-perfect usando Pandoc acoplado ao motor **Typst** e o template `templates/template_apostila.typ`.
 - *Regras:* Tamanho de arquivo estritamente inferior a 5MB, fontes vetoriais incorporadas (Inter e Poppins), suporte de texto extraível para o leitor, e inclusão de quebras de página lógicas.
- **SPEC_HTML.md (Contrato do HTML - Landing Page e Apresentação):**
 - *Como funciona:* Gera um arquivo HTML único, autocontido, inline (sem dependências externas de stylesheets ou scripts), responsivo e com design elegante baseado em gradientes.
 - *Regras de Apresentação:* Painéis de conteúdo devem ter pelo menos 32px de padding vertical. Listas com 4 ou mais marcadores devem ser divididas automaticamente em duas colunas paralelas balanceadas para otimizar o respiro visual da tela.
- **SPEC_ARTE.md (Contrato da Arte PNG):**
 - *Como funciona:* Gera código HTML especializado e renderiza uma imagem PNG em tamanho real simulando o navegador usando o compilador Playwright (`scripts/compilar-arte.py`).
 - *Regras:* Dimensões pixel-perfect exatas por variante (1080×1080 para feed quadrado, 1080×1350 para feed retrato, 1080×1920 para stories), tamanho final menor que 1MB, e exclusão opcional de elementos decorativos pesados através da chave `elementos_decorativos` no arquivo de configuração do projeto.
- **SPEC_KITS.md (Contrato dos Kits do Consultor e Distribuidor):**
 - *Como funciona:* Produz 10 artes PNG distintas no formato 1080×1350 combinando 5 tons visuais pré-definidos (Azul Classic, Azul Royal, Azul Teal, Azul Sky e Cinza Platina) vezes 2 templates de conteúdo (Foto-Produto e Texto-Conceitual).
 - *Regras:* As copies e textos de apoio das postagens de WhatsApp correspondentes são compartilhados de forma idêntica entre o Kit Consultor e o Kit Distribuidor, variando unicamente o foco da chamada para ação (CTA) e a assinatura final.

5.5. Scripts de Validação (Árbitros Determinísticos)

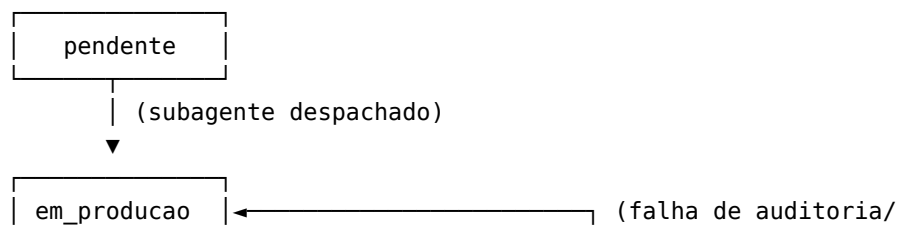
A qualidade estética e estrutural de cada material não é deixada à livre interpretação ou julgamento do assistente virtual (REGRA 8). Ela é governada por scripts python de validação estritos que fornecem gates de barreira objetivos:

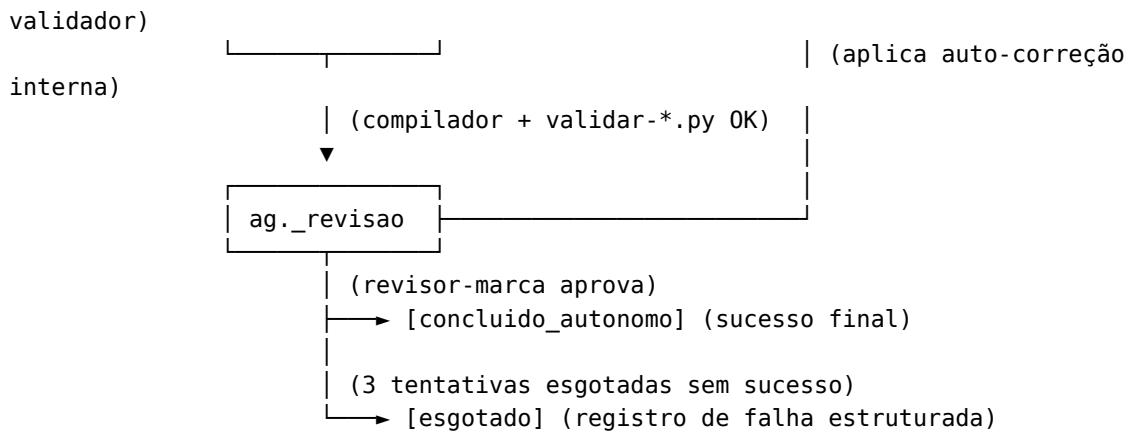
- `validar-html.py`: Verifica se o HTML gerado é perfeitamente semântico, se todos os componentes exigidos pelo DS Conexão estão com as classes CSS corretas, se não há links externos quebrados ou scripts que disparem exceções no console do navegador virtual (Playwright).
- `validar-pdf.py`: Analisa a assinatura estrutural do PDF compilado, garantindo o tamanho final menor que 5MB, a correta incorporação de fontes e a conformidade do fluxo de texto.
- `validar-dimensoes.py`: Checa as dimensões milimétricas de cada arquivo PNG exportado de acordo com seu slug e variante esperados.
- `validar-kit.py`: Garante que as 10 artes de cada kit foram compiladas corretamente seguindo a matriz de 5 tons × 2 layouts, além de testar a presença dos arquivos de texto e copies de WhatsApp.
- `validar-textos.py`: Analisa se as copies geradas para posts e textos de WhatsApp não contêm placeholders não preenchidos ou quebras de formatação de caracteres.
- `validar-logo.py`: Garante que os logos oficiais corretos (assets) foram corretamente injetados no rodapé dos templates sem perda de proporção ou quebras.
- `validar-transparencia.py`: Valida se o canal alfa (transparência) de imagens embutidas está sendo tratado corretamente no processo de compilação sem gerar bordas serrilhadas ou fundos brancos artificiais.
- `auditar-projeto.py`: O consolidador de conformidade final. Varre toda a estrutura física de diretórios do projeto final gerado contra as especificações registradas na fase /esbocar em `config_projeto.json`, gerando o `relatorio_auditoria.json`. Se houver qualquer falha estética ou estrutural, bloqueia o encerramento com exit code 1.

6. O Ciclo de Resiliência Agêntica (Máquina de Estados)

A resiliência contra falhas no processo de compilação ou rejeições estéticas é garantida por uma máquina de estados robusta gerida de forma persistente através do arquivo temporário `output/<slug>/_pool_estado.json`.

Cada material selecionado na entrevista de esboço inicia com o status de pendente. No momento em que o lote de orquestração do `pool-materiais.py` inicia seu processamento, as transições ocorrem de acordo com a conformidade dos gates determinísticos de validação:





Essa modelagem de estados persistente garante que, mesmo diante de uma interrupção da conexão de internet ou travamento do harness, a fábrica possa retomar o processamento exatamente do ponto em que parou, lendo as informações estruturadas de `_pool_estado.json` sem desperdício de tokens com materiais que já haviam obtido sucesso e conformidade na auditoria de gates.